WNDRVR
SUPPORT NETWORK

# VXWORKS 7 FTP PROGRAMMER'S GUIDE, SR0650

**Copyright Notice**

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

**Corporate Headquarters**

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

*VxWorks 7 FTP Programmer's Guide, SR0650*

10 June 2020

# 1. OVERVIEW

## 1. FTP for VxWorks

The VxWorks® 7 FTP Programmer's Guide describes the VxWorks File Transfer Protocol (FTP) for exchanging files over any TCP/IP network.

An FTP server listens on the network for connection requests from another computer, the FTP client, which connects to the FTP server using FTP client software. The FTP client can then manipulate files on the server, for instance uploading, downloading, renaming, and deleting them.

The FTP client and server support the Transport Layer Security (TLS) protocol.

The Wind River FTP implementation includes a client and server, and FTP components for backward compatibility. IPv4 and IPv6 are implemented in the same module.

### Source Code Files

The source code files for FTP are in the following directory:

*installDir* **/vxworks-7/pkgs_v2/net/ipnet/app/ftp-***x.x.x.x/*

**Next topic:** Supported RFCs for FTP on page 1

## 2. Supported RFCs for FTP

File Transfer Protocol (FTP) request for comments (RFCs) contain standards and design information defined by the Internet Engineering Task Force (IETF).

### Wind River FTP complies with the following IETF RFCs:

- RFC 959: *File Transfer Protocol*
  - Wind River FTP implements only the stream transmission mode of RFC 959, and not the block or compressed transmission modes.
- RFC 1123: *Requirements for Internet Hosts--Application and Support*
- RFC 2228: *FTP Security Extensions*
- RFC 2246: *The TLS Protocol*
- RFC 2428: *FTP Extensions for IPv6 and NAT*
- RFC 2577: *FTP Security Considerations*
- RFC 3659: *Extensions to FTP*
  - Wind River FTP implements section 4, File SIZE, supporting the SIZE command by the FTP server.

**Previous topic:** FTP for VxWorks on page 1

# 2. CONFIGURING AND BUILDING FTP

## 1. FTP Configuration and Build Workflow

Your Wind River VxWorks 7 installation includes source code but does not include pre-built kernel libraries.

Before beginning any VxWorks development, you must build the VxWorks kernel libraries (VSB project). You must also build the VxWorks run-time kernel image based on the kernel libraries (VIP).

For specific instructions on how to configure and build a VSB and a VIP, and how to create a downloadable kernel module (DKM) or a real-time process (RTP) application, see the VxWorks 7 Configuration and Build Guide.

For a list of available source configuration options for FTP, see FTP Source Build Configuration Options (VSB) on page 2.

For a list of available kernel configuration components and parameters for FTP, see FTP Kernel Configuration Components (VIP) on page 2.

**Next topic:** FTP Source Build Configuration Options (VSB) on page 2

## 2. FTP Source Build Configuration Options (VSB)

File Transfer Protocol (FTP) configuration includes VxWorks source build (VSB) options.

The table below lists FTP VSB options.

| Option | Description |
|---|---|
| IPNET_FTP | Enable the FTP client and server in the VSB. |
| IPFTP_USE_TLS | Enable Transport Layer Security (TLS) protocol for the FTP server or FTP client. See FTP Server and TLS Configuration Parameters on page 7 |
| USER_MANAGEMENT | Enable authentication. This component is required if you plan to use INCLUDE_SECURITY to configure authentication. See FTP Optional Kernel Configuration Components on page 3. |
| SEC_CRYPTO | Enable security. This component is required if you plan to use IPFTP_USE_TLS. See FTP Optional Kernel Configuration Components on page 3. |

**Previous topic:** FTP Configuration and Build Workflow on page 2
**Next topic:** FTP Kernel Configuration Components (VIP) on page 2

## 3. FTP Kernel Configuration Components (VIP)

File Transfer Protocol (FTP) configuration includes VxWorks image project (VIP) kernel configuration components.

The table below lists the required FTP VIP components for FTP functionality.

| Component | Description |
|---|---|
| INCLUDE_IPFTPC | Include the FTP client in your VIP. |
| INCLUDE_IPFTPS | Include the FTP server in your VIP.<br>See FTP Server Configuration Parameters on page 4. |
| INCLUDE_FTP | Include this component if you are configuring a system with the FTP client for IPv4 and require backward compatibility with a previous release of the Wind River Network Stack.<br>See FTP Client IPv4 Backend Configuration Parameters on page 7. |
| INCLUDE_FTP6 | Include this component if you are configuring a system with the FTP client for IPv6 and require backward compatibility with a previous release of the Wind River Network Stack.<br>See FTP Client IPv6 Backend Configuration Parameter on page 8. |

**Previous topic:** FTP Source Build Configuration Options (VSB) on page 2
**Next topic:** FTP Optional Kernel Configuration Components on page 3

# 4. FTP Optional Kernel Configuration Components

You can modify the File Transfer Protocol (FTP) client behavior with a number of optional shell command components.

The following table lists optional FTP components and shell command components.

| Component | Description |
|---|---|
| INCLUDE_IPFTPS_TLS | Include Transport Layer Security (TLS) in your VIP. This component requires INCLUDE_IPFTPS and INCLUDE_IPSSL.<br>See FTP Server and TLS Configuration Parameters on page 7. |
| INCLUDE_IPSSL | Include the Secure Socket Layer (SSL) library in the VIP.<br>To access the SSL shell commands, include INCLUDE_IPSSL_USE_CMDS.<br>To access the SSL test shell commands, include INCLUDE_IPSSL_USE_TEST_CMDS.<br>For information on SSL, see VxWorks 7 SSL Programmer's Guide. |
| INCLUDE_SECURITY | Include authentication for FTP. |

| Component | Description |
|---|---|
| | For information on installing a custom authentication hook, see Using a Client Authentication Routine with IPv6 and an FTP Server on page 19 |
| INCLUDE_BOOT_FTP_LOADER | Include the boot application to load files with FTP at startup.<br>For information on the bootFtpLoadInit( ) routine, refer to the VxWorks utils layer, specifically *installDir* **/ vxworks-7/pkgs_v2/os/utils-***x.x.x.x***/bootapp/src/ bootFtpLoad.c**. |
| INCLUDE_IPFTP_CMD | Include the FTP client shell command (ftp). |
| **INCLUDE_IPD_CMD** | Include the daemon control shell command (ipd). |
| INCLUDE_IPCOM_SYSLOGD_CMD | Include the system logging shell command (syslog). |
| **INCLUDE_IPCOM_SYSVAR_CMD** | Include the system variables shell command (sysvar). |

**Previous topic:** FTP Kernel Configuration Components (VIP) on page 2
**Next topic:** FTP Server Configuration Parameters on page 4

# 5. FTP Server Configuration Parameters

You can modify the Wind River File Transfer Protocol (FTP) server behavior with a number of configuration parameters and system variables.

The table below lists the FTP server configuration parameters and system variables.

| Parameter Name, System Variable Name, and Description | Default and Type |
|---|---|
| FTPS_CFGFILE<br>The full path of the configuration file that VxWorks will use to configure the FTP server parameters.<br>When no file is specified (default setting), the FTP server parameter values are based on the VIP configuration. | **""**<br>char * |
| FTPS_DELAYEDSTART<br>When set to **FALSE**, the FTP server is automatically started upon booting VxWorks.<br>When set to **TRUE**, the FTP server is not started upon booting VxWorks. The FTP server can be started with an **ipd** command. | **FALSE**<br>BOOL |
| FTPS_AUTH_ATTEMPTS ipftps.authentications<br>The number of times the server will allow a client to attempt to authenticate itself before the server disconnects. | **"3"**<br>char * |

| Parameter Name, System Variable Name, and Description | Default and Type |
|---|---|
| FTPS_AUTH_CALLBACK_HOOK<br>A user-defined authentication callback routine for the FTP server. The default callback routine is ipftps_authenticate_default_callback( ). | **ipftps_authenticate_ default_callback**<br>funcptr |
| FTPS_SOCKET_BACKLOG ipftps.socket_backlog<br>Determines the backlog queue length of the FTP server control socket. | **"0"**<br>char * |
| FTPS_RECV_TIMEOUT ipftps.receive_timeout<br>The timeout value in seconds when the server is receiving data from the client. If the server does not get any data from the client in this many seconds, the server terminates the connection. | **"30"**<br>char * |
| FTPS_SEND_TIMEOUT ipftps.send_timeout<br>The timeout value in seconds when the server is sending data to the client. If the server is unable to transmit any data to the client for this many seconds, the server terminates the connection. | **"30"**<br>char * |
| FTPS_ENABLE_PROXY ipftps.proxy<br>Specifies whether to enable proxy FTP. **"1"** means enable, **"0"** means disable. | **"0"**<br>char * |
| FTPS_INITIAL_DIR ipftps.dir<br>The directory on the server that a client starts in when it initiates an FTP session. If you use the default value, IPCOM_FILE_ROOT, you must also include the component INCLUDE_IPCOM_FS_NATIVE. | **IPCOM_FILE_ ROOT**<br>char * |
| FTPS_ROOT_DIR ipftps.root<br>The topmost directory that the client is allowed to see. | **""**<br>char * |
| FTPS_LOCAL_PORT_BASE ipftps.lportbase<br>Defines the port range the server uses for the data connection. A value of "0" instructs the server to use port 20. The default value of "49151" instructs the server to use ports 49152-65535. | **"49151"**<br>char * |

| Parameter Name, System Variable Name, and Description | Default and Type |
|---|---|
| FTPS_MAX_SESSIONS ipftps.max_sessions<br>The maximum number of simultaneous FTP sessions. | **"8"**<br>char * |
| FTPS_PEER_PORT_BASE ipftps.pportbase<br>The base of the port range that the client is allowed to use for PORT and EPRT commands when it establishes the data connection. Set this to "65535" to disable the PORT and EPRT commands entirely (which Wind River recommends for maximum security). The default value, "1023" prevents the server from making any connections to ports 0-1023 on a client. | **"1023"**<br>char * |
| FTPS_MODE ipftps.readonly<br>The access mode. Read-only mode disables write access to the file system, which means that only read commands will be accepted. The default value is **"0"**, which means read/write access. Set this to **"1"** for read-only access. | **"0"**<br>char * |
| FTPS_SYS_TYPE ipftps.system<br>The operating system name. Some FTP clients use the reported system type to determine how to work with the server to which they are connecting. Assign a string such as **"LINUX"** to identify the server operating system to the client. | **"UNIX"**<br>char * |
| FTPS_PORT_NUM ipftps.port_number<br>The port number the FTP server uses for the control connection. | **"21"**<br>char * |
| FTPS_SLEEP_TIME ipftps.authsleep<br>How many seconds the server sleeps before it replies to an unsuccessful authentication attempt. | **"5"**<br>char * |
| FTPS_INACTIVITY_TIMEOUT ipftps.session_timeout<br>The timeout in seconds when the server waits for a command from the client. If the timeout expires, the server terminates the connection. | **"300"**<br>char * |
| FTPS_SESSION_TASK_PRIORITY ipftps.session_priority<br>Sets the task priority of the FTP network service session. | **"50"**<br>char * |

| Parameter Name, System Variable Name, and Description | Default and Type |
|---|---|
| ipftps.dsock_linger<br>It is the FTPS data socket linger time in seconds. | **"10"**<br>char * |
| ipftps.versionstr<br>It is the FTPS version information. | **"6.8"**<br>char * |

**Previous topic:** FTP Optional Kernel Configuration Components on page 3
**Next topic:** FTP Server and TLS Configuration Parameters on page 7

# 6. FTP Server and TLS Configuration Parameters

The File Transfer Protocol (FTP) server Transport Layer Security (TLS) configuration parameters and system variables.

| Parameter Name, System Variable Name, and Description | Default and Type |
|---|---|
| ENABLE_ADVANCE_FTPS_KEY_PRIVACY ipftps.tls.private.enable<br>If the value is "1", the private key is saved to KEY DB. This protects the private key, preventing it from being accessed by others. If the value is "0", the private key is not saved to KEY DB. In order to use KEY DB, you must include INCLUDE_IPCOM_USE_RAM_DISK. | **"0"**<br>char * |
| FTPS_TLS ipftps.tls.enable<br>To use the FTP server with TLS, set this value to **"1"**,. To disable this feature, set the value to **"0"**. | **"0"**<br>char * |
| FTPS_TLS_CERT_FILE ipftps.tls.cert_file<br>Use this parameter to assign a certificate file. | **" "**<br>char * |
| **FTPS_TLS_KEY_FILE ipftps.tls.key_file**<br>Use this parameter to assign a private key file. | **" "**<br>char * |

**Previous topic:** FTP Server Configuration Parameters on page 4
**Next topic:** FTP Client IPv4 Backend Configuration Parameters on page 7

# 7. FTP Client IPv4 Backend Configuration Parameters

The File Transfer Protocol (FTP) client IPv4 backend configuration has a number of parameters.

| Parameter Name and Description | Default and Type |
|---|---|
| FTP_DEBUG_OPTIONS<br>Enable various debugging facilities within **ftpLib**. Set to 1 to enable debugging or set to 0 to disable debugging. | **0**<br>int |
| FTP_TRANSIENT_MAX_RETRY_COUNT<br>Maximum number of retries when an FTP_TRANSIENT response is encountered. | **100**<br>int |
| FTP_TIMEOUT<br>FTP timeout in seconds. Timeout is the amount of time FTP waits for a new connection started from a remote point. If it the timeout counter expires, the command fails.<br>If the default value of zero is assigned, FTP uses an internal default of 10 sec. If any nonzero value is assigned, FTP uses that value. | **0**<br>long |
| FTP_TRANSIENT_FATAL<br>The default value is ftpTransientFatal( ). To establish a connection between an FTP client and an FTP server, call ftpXfer( ). ftpXfer( ) retries a command if the reply packet from the host contains a **4xx** code. If the VxWorks image includes ftpTransientFatal( ), it can immediately terminate the retry sequence. ftpTransientFatal( ) is defined in **usrFtp.c**. | ftpTransientFatal( ) |
| FTP_TRANSIENT_RETRY_INTERVAL<br>The time interval (in clock ticks) between reissuing a failed command. If a value of zero is assigned, FTP immediately reissues the command without waiting. If a nonzero value is assigned, FTP waits the designated interval before reissuing the failed command. | **0**<br>int |

**Previous topic:** FTP Server and TLS Configuration Parameters on page 7
**Next topic:** FTP Client IPv6 Backend Configuration Parameter on page 8

# 8. FTP Client IPv6 Backend Configuration Parameter

File Transfer Protocol (FTP) IPv6 client backend has a configuration parameter.

The table below lists the FTP IPv6 client backend configuration parameter.

| Parameter Name and Description | Default and Type |
|---|---|
| **FTP6_REPLYTIMEOUT**<br>The timeout in seconds, while waiting for a reply from the FTP server. For IPv6 systems only. | **10**<br>long |

**Previous topic:** FTP Client IPv4 Backend Configuration Parameters on page 7

# 3. FILE-BASED CONFIGURATION

## 1. About FTP File-Based Configuration

By default, File Transfer Protocol (FTP) server parameter values are set at VxWorks image project (VIP) configuration time. The file-based configuration functionality allows you to reconfigure FTP parameter values at boot time.

You reconfigure FTP server parameter values with a list of system variables stored in a text-based configuration file. The file path and file name are specified by a VIP parameter. This configuration method provides an easy way to adapt a system to new networking environments by simply updating the configuration file contents and then transferring the file to the target before starting the FTP server.

A delayed start functionality is also available to prevent the FTP server from starting automatically during the VxWorks boot process. The delayed start functionality can be used to explicitly start the FTP server after VxWorks has booted or at runtime from your application.

**Next topic:** Enabling and Disabling FTP File-Based Configuration on page 10

## 2. Enabling and Disabling FTP File-Based Configuration

To enable the configuration of the File Transfer Protocol (FTP) server from a configuration file, you must specify the full path to the configuration file as part of the FTPS_CFGFILE VxWorks image project (VIP) parameter.

To disable the configuration of the FTP server from a configuration file, set the FTPS_CFGFILE VIP parameter to NULL (**""**). The FTP server configuration is then based on the VIP configuration.

For information on configuring the FTPS_CFGFILE VIP parameter, see FTP Server Configuration Parameters on page 4.

For information on startup scenarios for the FTP server, see FTP Server Startup Scenarios on page 13.

For information on operational requirements for the target and FTP server, see FTP Operational Requirements on page 14.

**Previous topic:** About FTP File-Based Configuration on page 10
**Next topic:** Sample FTP Server Configuration File on page 10

## 3. Sample FTP Server Configuration File

Here is a sample File Transfer Protocol (FTP) configuration file that contains the list of valid system variables that can be configured from a configuration file.

The sample configuration file is at:

*installDir* **/vxworks-7/pkgs_v2/net/ipnet/app/ftp-***x.x.x.x***/sample_cfg/etc/ ftps.conf**

> Note: Only the specified system variables in the sample file are configurable from a configuration file.

To correlate FTP VIP parameter names to FTP system variables listed in the sample configuration file, see FTP Server Configuration Parameters on page 4.

Note: To prevent unnecessary reconfiguration of the VIP parameters, the configuration file should only contain system variables that must be changed. The remaining parameter values will be taken from the VIP configuration.

**Previous topic:** Enabling and Disabling FTP File-Based Configuration on page 10
**Next topic:** FTP Configuration File Format on page 11

# 4. FTP Configuration File Format

The File Transfer Protocol (FTP) configuration file is a text-based file you can edit with a standard text editor.

To configure a parameter from a configuration file, a valid system variable name must be followed by a valid value (system variable / value pair) and be on a single line.

A configuration file can also contain comments.

Note: The configuration file does not accept Escape characters (\).

## System Variable and Value Pair

A system variable/value pair consists of a system variable name and a value. Valid system variables are provided in the sample configuration file.

A system variable/value pair can start in any column of a given line. A system variable and name must be separated by least one space or tab.

Each line can only contain a single system variable/value pair.

Boolean, integer, floating point, and string values can be listed with or without double quotation marks. Example valid line content follows:

```
systemVariableName_A TRUE
systemVariableName_B 2
systemVariableName_C 192.168.1.1
systemVariableName_D String

systemVariableName_A "TRUE"
systemVariableName_B "2"
systemVariableName_C "192.168.1.1"
systemVariableName_D "String"
```

## Comment Delimiter

The comment delimiter for the configuration file is the pound (**#**) character. For example:

```
# This is a comment line.
```

You can add a comment on the same line as a system variable/name pair. For example:

```
systemVariableName_A FALSE    # This is a boolean value.
systemVariableName_B 22    # This is an integer value.
```



WNDRVR

```
systemVariableName_C 192.168.2.2    # This is a floating point value.
systemVariableName_D String    # This is a string value.
```

**Previous topic:** Sample FTP Server Configuration File on page 10
**Next topic:** FTP Configuration File Processing on page 12

# 5. FTP Configuration File Processing

All empty lines and comment lines are not processed.

If a configuration file is not found, no processing is done and the system variables retain their values from the VIP configuration.

If a configuration file is located, the system processes the configuration file line-by-line. The values in the configuration file overwrite the values set at VIP configuration time.

Lines with errors such as misspelled system variable names and missing parameter values are ignored (skipped). Hence, a system variable associated with a faulty line entry retains the value set at VIP configuration time.

Note: If a system variable/value pair is listed more than once in a file, the last valid occurrence is used to configure the system variable. No error is reported as part of the diagnostic output.

**Previous topic:** FTP Configuration File Format on page 11
**Next topic:** FTP Diagnostics on page 12

# 6. FTP Diagnostics

You enable and disable diagnostics from within the configuration file. When enabled, the configuration status of the system variables is sent to the console, if a console is available.

To enable diagnostics, set the **cfgfile.diags** variable in the configuration file to **TRUE**. To disable, set to **FALSE**. For example:

```
cfgfile.diags      FALSE
```

When enabled, the diagnostics process generates output and provides status on the action performed on each system variable.

An updated system variable is labelled as **Updated**. The output format is as follows:

```
Variable: systemVariableName Value: value Result: Updated
```

A skipped update for a system variable is labelled as **Unrecognized variable (Skipping)**. The output format is as follows:

```
Variable: systemVariableName Value: value Result: Unrecognized variable (Skipping)
```

**Previous topic:** FTP Configuration File Processing on page 12
**Next topic:** Verifying FTP System Variables on page 13

# 7. Verifying FTP System Variables

To verify the configuration of the system variables, you can display the system variables and their content from the command shell.

### About This Task

To view system variables, use the following command:

```
[vxWorks *]# sysvar list serviceName
```

Note: To access the sysvar shell command, you must add the INCLUDE_IPCOM_SYSVAR_CMD component to your VIP.

**Previous topic:** FTP Diagnostics on page 12
**Next topic:** FTP Server Startup Scenarios on page 13

# 8. FTP Server Startup Scenarios

File Transfer Protocol (FTP) startup scenarios may require that configure parameters be set.

The table below lists the possible startup scenarios as set by the FTPS_CFGFILE and FTPS_DELAYEDSTART VIP configuration parameters.

| FTPS_CFGFILE VIP Parameter Setting | FTPS_DELAYEDSTART VIP Parameter Setting | Operation |
|---|---|---|
| Null file path | FALSE | The FTP server starts automatically at boot time.<br>FTP server parameter values are based on the VIP configuration.<br>This is the default configuration. |
| Valid file path | FALSE | The FTP server starts automatically at boot time.<br>FTP server parameter values are reconfigured based on valid system variable names and values listed in the configuration file. |
| Null file path | TRUE | The FTP server is not started at boot time.<br>The FTP server can be started with an ipd start command executed from a startup script or programmatically.<br>FTP server parameter values are based on the VIP configuration. |
| Valid file path | TRUE | The FTP server is not started at boot time. |



WNDRVR

| FTPS_CFGFILE VIP Parameter Setting | FTPS_DELAYEDSTART VIP Parameter Setting | Operation |
|---|---|---|
| | | The FTP server can be started with an ipd start command executed from a startup script or programmatically.<br>FTP server parameters are reconfigured based on valid FTP system variable names and values listed in the configuration file. |

Note: To access the ipd shell command, you must add the INCLUDE_IPD_CMD component to your VIP.

**Previous topic:** Verifying FTP System Variables on page 13
**Next topic:** FTP Operational Requirements on page 14

# 9. FTP Operational Requirements

To enable parameter configuration from a file, the target hosting the File Transfer Protocol (FTP) server must include read/write mass storage capability.

A configuration file can be modified by an external host and transferred to the target using one of the following methods:

- By transferring the file from a FTP server to the target.
- By mounting the target's mass storage device as a USB disk and using desktop system facilities to update the configuration file.
- By using a removable mass storage device supported by the target.

**Previous topic:** FTP Server Startup Scenarios on page 13

# 4. SHELL COMMANDS

## 1. About FTP Shell Commands

The File Transfer Protocol (FTP) shell commands include ftp, sysvar, ipd and syslog.

You can use the ftp shell command to configure and run the FTP client.

You can use the sysvar shell command to modify FTP configuration parameters.

You can use the ipd shell command to start and stop the FTP server daemon.

You can use the syslog shell command to set the priority for the FTP syslog debug messages.

To run shell commands, you must access the command-interpreter mode. To access the command-interpreter mode from the kernel shell, enter the following command:

```
-> cmd
```

[vxWorks *]#

To return to the kernel shell, enter the following command:

```
# C
```

->

**Next topic:** About FTP Client Shell Commands on page 15

## 2. About FTP Client Shell Commands

You can configure the FTP client at start time.

To configure the FTP client at start time, use the following command syntax:

```
ftp [-a family] [-C]
[-c file] [-i] [-k file]
[-l base] [-r base]
[-p pMode] [-P port]
[-s C|P] [-t] [-v vMode]
[-x xMode] host
```

The available configuration options for the FTP client are defined in FTP Client Configuration Options on page 22.

To launch the FTP client with the default configuration options, enter the following command:

```
# ftp
ftp>
```

To view the current configuration of the FTP client, use the **status** command. For example (default configuration shown):

```
ftp> status
status
Interpeak FTP client, version 1.1.3
Not connected
Allowed address families: IPv4 and IPv6
Connect to server port 21
Passive mode: locked
Extended mode: on (fallback to non-extended
mode)
IP address match for control and data: on
Client OS selects data port
Require server data ports above 1023
Verbose level: 2
Command-line editing: on
```

To list the available FTP commands, use the following command:

```
ftp> help
```

The get help for a command, enter **help** followed by the command name. For example:

```
ftp> help family
help family
 family [0|4|6] -- Specify address family: Any|IPv4|IPv6
```

For more information, including help for the commands, see A.3 Using the FTP Client Shell Commands, p. 30.

To return to the command-interpreter mode, enter the following command:

```
ftp> quit
#
```

**Previous topic:** About FTP Shell Commands on page 15
**Next topic:** Setting the FTP Server System Variables on page 16

# 3. Setting the FTP Server System Variables

You can view all File Transfer Protocol (FTP) server sysvar variables from the kernel shell in command-interpreter mode.

To view all FTP sysvar variables, access the command-interpreter mode from the kernel shell and execute the following command:

```
# sysvar list ipftps
```

To change FTP parameters with sysvar, use the sysvar set command.

**Previous topic:** About FTP Client Shell Commands on page 15
**Next topic:** Stopping and Starting the FTP Server Daemon on page 17

# 4. Stopping and Starting the FTP Server Daemon

Use the ipd command to stop or start the FTP server daemon (ipftps).

For example:

```
# ipd list
Services:
...
ipftps started

# ipd kill ipftps
ipd: kill ipftps ok

# ipd list
Services:
...
ipftps killed

#ipd start ipftps
ipd: start ipftps ok
```

To view the task information for the FTP server, run the task command and locate the ipftps task. For example:

```
# task
NAME    ENTRY        TID       PRI  STATUS    PC        SP        ERRNO    DELAY
...
ipftps ipftps_main  106233c8  50   PEND      102f41db  10fbfd90  0        0
```

**Previous topic:** Setting the FTP Server System Variables on page 16
**Next topic:** Setting the FTP System Log Priority on page 17

# 5. Setting the FTP System Log Priority

Run the syslog list command to get the current priority level for all modules.

```
# syslog list
   syslog facility  priority
   ...
   ipftp Error
```

To list the available syslog priority levels, use the following command:

```
# syslog priority *
syslog: missing <prio>
        priorities: Emerg Crit Error Warning Notice Info Debug Debug2
[vxWorks *]#
```

Use the following command syntax to change the priority level for FTP:

```
# syslog priority ipftp priorityLevel
```



WNDRVR

For more information on the syslog shell command, see the VxWorks 7 Real-Time Network Stack Programmer's Guide.

**Previous topic:** Stopping and Starting the FTP Server Daemon on page 17

# 5. API FUNCTIONS

## 1. FTP Client API Functions

The table below lists the API functions available to FTP client applications and gives a brief description of each routine. For more information, see the reference entries for the FTP client (ipftp).

The FTP client public API functions are defined in **ipftpc.c**.

| Routine | Description |
|---|---|
| ipftpc_close( ) | Close a connection to an FTP server. |
| ipftpc_cmd( ) | Send a command to the FTP server and wait for a reply. |
| ipftpc_get( ) | Get a file from the FTP server and wait for a reply |
| ipftpc_getattr( ) | Get the current set of session attributes. |
| ipftpc_lastreply( ) | Get the last server output processed by the FTP client |
| ipftpc_list( ) | Send a list command to the FTP server and wait for a reply. |
| ipftpc_login( ) | Log in to an FTP server. |
| ipftpc_open( ) | Open a connection to an FTP server. |
| ipftpc_put( ) | Send a file to the FTP server and wait for a reply |
| ipftpc_setattr( ) | Set attributes for a new session. |
| ipftpc_strerror( ) | Get a textual description of an error. |
| ipftpc_strclass( ) | Get a text description of an error class. |

**Next topic:** Using a Client Authentication Routine with IPv6 and an FTP Server on page 19

## 2. Using a Client Authentication Routine with IPv6 and an FTP Server

Once authentication is enabled, you can either use the default authentication routine or you can install a custom authentication hook. The callback routine is specified by the **Authentication callback routine** parameter.

### About This Task

For information on using the default authentication routine, see the implementation of ipftps_authenticate_default_callback( ) in **ipftps_example.c**. See also FTP Server Configuration Parameters on page 4.

When you configure the kernel for IPv6 and include the FTP server component and define a custom authentication routine, you must define IPCOM_USE_INET6 in your implementation of the authentication routine to include the IPv6 data structures. To include the IPv6 data structures, you must add the following statements in your source code before the #include command for the ipcom_sock.h( ) header file:

```
#ifdef INCLUDE_IPCOM_USE_INET6
#ifndef IPCOM_USE_INET6
#define IPCOM_USE_INET6
#endif
#endif
...
#include <ipcom_sock.h>
```

The example below shows how to implement a custom client authentication routine.

### Custom Authentication Routine for IPv6-Enabled Kernels

```
/*********************************************************************
User defined ftps authenticate hook
*********************************************************************/

#ifdef INCLUDE_IPCOM_USE_INET6
#ifndef IPCOM_USE_INET6
#define IPCOM_USE_INET6
#endif
#endif

#include <ipcom_type.h>
#include <ipcom_sock.h>
#include "ipftps_config.h"
#include <ipftps.h>

/*************************************************************************
* * ipftps_authenticate_customized_callback - customized ftps callback routine
*/

int ipftps_authenticate_customized_callback(Ipftps_session *session, char
*password)
{
  /*
  ** This is a simple authentication skeleton,
  ** modify it to suit your purposes.
  */

if (strcmp(session->username, "anonymous")== 0)
 {
   /* IPCOM_LOG1(INFO, "Anonymous login with password %s", password); */
   /* Access restrictions apply for anonymous users */
   session->readonly = 1;
 }

else
 {
  /* user customized authentication process */
 }
```

```
return 0;
}
```

**Previous topic:** FTP Client API Functions on page 19

# 6. FTP CLIENT SHELL COMMANDS

## 1. FTP Client Configuration Options

The File Transfer Protocol (FTP) client command allows you to transfer files using the FTP protocol.

To start the FTP client and configure the client options at startup, use the following command syntax:

```
ftp [-a family][-C] [-cfile]
[-i] [-kfile][-l base] [-r base]
[-p pMode] [-P port] [-sC|P ] [-t]
[-vMode] [-x xMode] host
```

To launch the FTP client with the default configuration options, enter the following command at the kernel shell prompt:

```
# ftp
ftp>
```

The available options for the ftp command are as follows:

**-a** *family*

Set the address family to use when connecting to a server. The default is 0. The options are as follows:

**0** - IPv4 or IPv6

**4** - IPv4 only

**6** - IPv6 only

**-C**

Enable clearing control connection (CCC). If this setting is enabled, the FTP client connects to the server, negotiates a secure connection, authenticates (sends user and password), and reverts back to plaintext.

**-c** *file*

Specify the certificate file.

**-i**

Disable IP address match for control and data. Without this flag, the control and data connection are required to have the same IP address (which disables proxy FTP).

**-k** *file*

Specify the private key file.

**-l** *base*

Set the lower limit for the client data port number. For instance, a value of 1023 means to use a random port in the range 1024-65535. A value of 0 will let the client operating system select a port. The default value is 0.

**-r** *base*

Set the lower limit for server data port number. A value of 1023 will only allow server ports in the range 1024-65535. The default value is 1023.

**-p** *pMode*

Select passive or active mode.

In active mode, the FTP client connects from a random unprivileged port greater than 1023 to the FTP server's command port 21. Then, the client starts listening to port N+1 and sends the FTP command PORT N+1 to the FTP server. The server then connects back to the client's specified data port from its local data port 20.

In passive mode FTP the client initiates both connections to the server, solving the problem of firewalls filtering the incoming data port connection to the client from the server. When opening an FTP connection, the client opens two random unprivileged ports locally (N > 1023 and N+1). The first port contacts the server on port 21, but instead of then issuing a PORT command and allowing the server to connect back to its data port, the client issues the PASV command. The server then opens a random unprivileged port (P > 1023) and sends the PORT P command back to the client. The client then initiates the connection from port N+1 to port P on the server to transfer data.

The default value is 3, but reverts to 1 if the server does not use extended mode. The options are as follows:

**0** - Use only active mode.

**1** - Try passive mode, but revert to active mode if passive mode is not allowed.

**2** - Use only passive mode.

**3** - Lock the server in extended passive mode, which means that the server will require that extended passive mode is used for all future data connections.

**-P** *port*

Set the server port number.

**-s C**| **P**

Set data session protect mode.

Setting **ftp -s C** means that the packets on the data connection are not encrypted. Setting **ftp -s P** means that the packets on the data connection are encrypted.

**-t**

Enable Transport Level Security (TLS) for the ftp client. The VIP project must have been based on a VSB that set the **Enable FTP with TLS** configuration option ( **IPFTP_USE_TLS**) to yes. To enable the FTP server, use the commands documented in 2.10 FTP Server Transport Layer Security Configuration Parameters, p. 8. The settings for the FTP client and FTP server are independent and are enabled separately.

**-v** *vMode*

Set the verbosity level, which controls the detail of console messages. The default value is 2.

**0** - brief mode

**1** - verbose mode

**2** - verbose mode + server output

**3** - verbose mode + client and server output

**-x** *xMode*

Select the extended mode commands EPRT (extended port) and EPSV (extended passive) for IPv4 connections.

The EPRT mode allows for the specification of an extended address for the data connection. The extended address MUST consist of the network protocol as well as the network and transport addresses.

The EPSV mode requests that a server listen on a data port and wait for a connection. The EPSV mode takes an optional argument. The response to this command includes only the TCP port number of the listening connection. The format of the response, however, is similar to the argument of the EPRT command. The format leaves a place holder for the network protocol and/or network address, which may be needed in the EPSV response in the future.

The default value is 1.

**0** - Never use extended commands.

**1** - First try extended commands, but fall back to normal if those are not allowed.

**2** - Always use extended commands.

*host*

The host name or IP address of the FTP server.

**Next topic:** Using the FTP Client Shell Commands on page 24

# 2. Using the FTP Client Shell Commands

Many File Transfer Protocol (FTP) client shell commands are available.

To list the available FTP client commands, use the following command:

```
ftp> help
```

The get help for a command, enter help followed by the command name.

The available FTP client commands are as follows:

append *local-file* [*remote-file* ]

Append local-file to remote-file on server.

ascii

Select ASCII mode for file transfer.

binary

Select binary mode for file transfer.

cd *remote-dir*

Change server working directory to remote-dir.

*remote-dir*

cdup

Change to parent server working directory.

close

Close connection to FTP server.

delete *remote-file*

Delete remote-file on server.

dir

List file attributes in remote-path on server.

extended [*mode* ]

Extended mode for IPv4:

The options are as follows:

**0**: Off

**1**: On -> Off

**2**: On.

family [*address* ]

Specify the address family:

The options are as follows:

**0**: Any

**4: IPv4**

**6: IPv6**

get *local-file* [*remote-file* ]

Get remote-file from server and store in local-file.

help [ *command_name* ]

Get command list, or help on command-name.

ipcheck [*state* ]

Require control and data address match:

**0**: OFF

**1**: ON

lbase [ *port* ]

Specify lower limit for client data port number. The range is 0 to 65535.

lcd *local-dir*

Change the client working directory to local-dir.

lpwd

Print client working directory.

ls [*remote-path* ]

List file names in remote-path on server.

mkdir *remote-dir*

Make directory remote-dir on server.

open *server* [*port* ]

Open connection to FTP server.

passive [*mode* ]

Passive mode: 0: Active1: Passive -> Active

2: Passive

3: Lock Passive

port [*port* ]

Select a server port. The range is 1 to 65535.

put *local-file* [*remote-file* ]

Put local-file to remote-file on server.

pwd

Print server working directory.

quit

End FTP session.

quote *command* ...

Send verbatim command to remote server. The maximum number of arguments is set by IPFTPC_MAX_ARGS -1.

rbase [*port* ]

Specify the lower limit for the server data port number. The range is 1 to 65535.

rename *old-file new-file*

Rename old-file to new-file on the server.

rhelp [*command-name* ]

Get command list, or help on command-name from the remote server.

rmdir *remote-dir*

Remove directory from the server.

rstatus [*remote-file* ]

Get server status, or status of remote-file.

status

Show current FTP status.

system

Show server operating system.

verbose [*level* ]

Verbose level:

**0**: Brief

**1**: Verbose

**2**: Server

**3**: Client

**Previous topic:** FTP Client Configuration Options on page 22